

give me a 5 day plan for learning python for my backend and its basics

5-Day Python Plan for Backend Development + Basics

Since you're already learning frontend + APIs, this plan focuses on **practical Python for backend**, not unnecessary theory.

Goal after 5 days:

- Understand Python basics
- Build logic confidently
- Understand backend concepts
- Create simple API/backend projects later using frameworks like FastAPI or Flask

Day 1 — Python Fundamentals

Topics

- Installing Python
- VS Code setup
- Variables
- Data types
- Input/output
- Operators
- Strings
- Type conversion

Learn These

```
name = "Yokii"  
age = 22  
is_student = True
```

</> Python



▶ Run

Input

```
name = input("Enter your name: ")
```

</> Python



▶ Run

Type Conversion

</> Python



▶ Run

Practice

Build:

- Simple calculator
- Age checker
- Even/odd checker

Important Concepts

- Python is indentation based
- No semicolons
- Very beginner friendly

Day 2 — Conditions + Loops + Functions

Topics

- if/else
- for loop
- while loop
- functions
- return keyword

Learn These

Conditions

```
age = 18

if age >= 18:
    print("Adult")
else:
    print("Minor")
```

Loops

```
for i in range(5):
```

</> Python



▶ Run

Functions

```
def greet(name):
    return f"Hello {name}"
```

</> Python



▶ Run

Practice

Build:

- Multiplication table
- Number guessing game
- Password checker

Day 3 — Lists, Dictionaries & Backend Data Thinking

This is VERY important for backend.

Topics

- Lists
 - Dictionaries
 - Nested data
 - Looping through data
-

Learn These

List

```
users = ["Ram", "Sam", "John"]
```

</> Python



▶ Run

Dictionary

```
user = {  
    "name": "Yokii",  
    "age": 22  
}  
  
print(user["name"])
```

Looping Data

```
users = [  
    {"name": "A"},  
    {"name": "B"}  
]  
  
for user in users:  
    print(user["name"])
```

Why Important for Backend?

Backend APIs usually return JSON like this:

```
{  
  "name": "Yokii",  
  "email": "test@gmail.com"  
}
```

Python dictionaries work almost exactly like JSON.

Practice

Build:

- Student marks manager
- Todo list
- User data system

Day 4 — File Handling + Error Handling + Modules

Backend developers MUST know this.

Topics

- Reading files
- Writing files
- try/except
- modules
- pip

Learn These

File Handling

```
file = open("test.txt", "w")  
file.write("Hello")
```

</> Python



▶ Run

Error Handling

```
try:
    num = int(input("Enter: "))
    print(num)
except:
    print("Invalid input")
```

Modules

```
import math
```

</> Python



▶ Run

Learn About

- pip
- virtual environments
- requirements.txt

Practice

Build:

- Notes app
- Login system
- File-based todo app

Day 5 — Backend Introduction (MOST IMPORTANT)

Now connect Python with backend.

Learn

- What is backend?

- What is API?
 - GET/POST requests
 - JSON
 - HTTP methods
-

Start With FastAPI

Use FastAPI because:

- Modern
 - Easy
 - Best for beginners
 - Used professionally
 - Automatic API docs
-

Install

`</> Bash`



First Backend API

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello World"}
```

Run:

`</> Bash`



Learn These Concepts

- Routes
 - APIs
 - Request
 - Response
 - JSON
 - Status codes
-

BEST YouTube Channels

Python Basics

- Programming with Mosh
 - Bro Code
 - CodeWithHarry
-

Backend Python

- Tech With Tim
 - freeCodeCamp
 - FastAPI Official Tutorials
-

What To Do After These 5 Days

After finishing:

1. Build REST API
 2. Connect MongoDB
 3. Learn authentication
 4. Build full backend project
 5. Connect React + Python backend
-

BEST Beginner Backend Projects

Easy

- Todo API
- Notes API
- URL Shortener

Medium

- Authentication system
- AI Chat backend
- Expense tracker API

Strong Resume Project

- SaaS AI backend
- Chat application
- File upload system
- Real-time notification backend

Daily Study Time Recommendation

Time	Work
1 hour	Learn concepts
2 hours	Coding
1 hour	Build mini project

Total:

- 3–4 hours daily is enough
-

Important Advice

Do NOT:

- Watch tutorials endlessly
- Memorize syntax
- Jump into Django immediately

Do:

- Build tiny projects daily
- Practice errors/debugging
- Use APIs
- Read documentation slowly

Python becomes easy once you build small real things consistently for 1–2 weeks.



ccess to fetch at 'http://127.0.0.1:8000/todos' from origin 'http://localhost:5173' has been blocked by CORS policy: Response to preflight request doesn't pass access control check: No 'Access-Control-Allow-Origin' header is present on the requested resource.

Cancel

Send

The error happens because:

```
useEffect(() => {  
  fetchTodos();
```

⌘ TypeScript



is trying to use `fetchTodos` **before it is declared**.

In JavaScript/TypeScript:

⌘ TypeScript



is NOT hoisted like normal functions.

Fix 1 (BEST)

Move `fetchTodos` ABOVE `useEffect` .

```
import { useEffect, useState } from "react";  
  
type Post = {  
  text: string;  
};  
  
function Todos() {  
  const [todos, setTodos] = useState<Post[]>([]);  
  const [input, setInput] = useState("");
```

```
// Fetch todos
const fetchTodos = async () => {
  const response = await fetch("http://127.0.0.1:8000/todos");

  const data = await response.json();

  setTodos(data);
};

useEffect(() => {
  fetchTodos();
```

VERY IMPORTANT FIX

You had:

</> TypeScript



This is wrong.

Correct:

</> TypeScript



Another BIG Problem

You wrote:

```
useEffect(() => {
  fetchTodos();
```

</> TypeScript



This creates an INFINITE LOOP.

Why?

1. fetchTodos() runs
2. setTodos(data) updates state

3. `todos` changes
 4. `useEffect` runs again
 5. repeat forever
-

Correct Dependency

Use:

```
</> TypeScript
```



Empty array means:

- run only once when component loads
-

Beginner Understanding

```
[]
```

```
</> TypeScript
```



Run once on page load.

```
[todos]
```

```
</> TypeScript
```



Run every time `todos` changes.

Why Your App Logic Works

Initial Load

```
useEffect(() => {  
  fetchTodos();  
})
```

⌘ TypeScript



Fetch todos when page opens.

Add Todo

⌘ TypeScript



after POST refreshes list.

Delete Todo

⌘ TypeScript



after DELETE refreshes list.

Extra Beginner Tip

Instead of:

```
type Post = {  
  text: string;
```

⌘ TypeScript



You can rename it better:

```
type Todo = {  
  text: string;
```

⌘ TypeScript



Then:

</> TypeScript



Cleaner naming.



↳ fetchTodos();

Error: Calling `setState` synchronously within an effect can trigger cascading renders

Effects are intended to synchronize state between React and external systems such as manually updating the DOM, state management libraries, or other platform APIs. In general, the body of an effect should do one or both of the following:

- * Update external systems with the latest state from React.
- * Subscribe for updates from some external system, calling `setState` in a callback function when external state changes.

Calling `setState` synchronously within an effect body causes cascading renders that can hurt performance, and is not recommended. (<https://react.dev/learn/you-might-not-need-an-effect>).

D:\Project\2026\python\todo-react-
python\frontend\src\components\Todo.tsx:21:5

```

19 |
20 |   useEffect(() => {
> 21 |     fetchTodos();
    |     ^^^^^^^^^^^^^ Avoid calling setState() directly
    |     within an effect
22 |   }, [todos]);
23 |
24 |   // Add todoeslintreact-hooks/set-state-in-effect
const fetchTodos: () => Promise<void>

```

The problem is STILL this:

```
useEffect(() => {  
  fetchTodos();
```

⌄ TypeScript



Because:

⌄ TypeScript



contains:

⌄ TypeScript



So React sees:

1. `useEffect` runs
2. `fetchTodos()` runs
3. `setTodos(data)` updates state
4. `todos` changes
5. effect runs AGAIN
6. infinite re-render loop

Correct Fix

Change this:

⌄ TypeScript



to this:

⌄ TypeScript



Final Correct Version

```
useEffect(() => {  
  fetchTodos();  
})
```

TS TypeScript



Why [] Works

TS TypeScript



means:

👉 Run only ONCE when component mounts.

Perfect for:

- API calls
- Initial data loading
- Authentication check

Why [todos] Fails

TS TypeScript



means:

👉 Run every time `todos` changes.

But inside effect:

TS TypeScript



changes `todos` .

So it loops forever.

Beginner Mental Model

Think like this:

```
useEffect(() => {  
  // DO SOMETHING
```

⌞ TypeScript



React says:

"Whenever `watch_this` changes,
run this effect again."

Your code was basically:

```
Whenever todos changes:  
  fetchTodos()  
  
fetchTodos():  
  setTodos()
```

So:

```
todos changes  
→ fetchTodos  
→ setTodos  
→ todos changes  
→ fetchTodos  
→ repeat forever
```

Your Correct Structure

```
const fetchTodos = async () => {  
  const response = await fetch("http://127.0.0.1:8000/todos");  
  
  const data = await response.json();
```

```
    setTodos(data);  
  };  
  
  useEffect(() => {  
    fetchTodos();  
  }, []);
```

This is now correct React logic.



ccess to fetch at 'http://127.0.0.1:8000/todos' from origin 'http://localhost:5173' has been blocked by CORS policy: Response to preflight request doesn't pass access control check: No 'Access-Control-Allow-Origin' header is present on the requested resource.

This is a VERY common backend/frontend issue.

Your frontend:

http://localhost:5173



is trying to access backend:

http://127.0.0.1:8000



Browser blocks it for security.

This is called:

CORS

Cross-Origin Resource Sharing

Fix in FastAPI Backend

Add CORS middleware.

Step 1

Open your FastAPI backend file.

Usually:

main.py



Step 2

Add this import

</> Python



▶ Run

Step 3

Add this BELOW:

</> Python



▶ Run

Full Correct Example

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

# CORS
origins = [
    "http://localhost:5173",
]

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
```

```
        allow_methods=["*"],
        allow_headers=["*"],
    )

    todos = []

    @app.get("/todos")
    def get_todos():
        return todos

    @app.post("/todos")
    def add_todo(todo: dict):
        todos.append(todo)
        return {"message": "Todo added"}
```

Step 4

Restart backend server.

IMPORTANT.

Stop server:

</> Bash



Then run again:

</> Bash



Why This Happens

Frontend and backend are on DIFFERENT origins.

Frontend

Backend

localhost:5173

127.0.0.1:8000

Browser says:

"This could be dangerous."

Backend must explicitly allow it."

CORS middleware adds permission headers.

Beginner Understanding

Without CORS:

Frontend ❌ Backend



With CORS:

Frontend ✅ Backend



Temporary Development Shortcut

You can allow ALL origins during learning:

Python



Run

Example:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

BUT:

❌ Not recommended in production.

One More Thing

You are mixing:

`localhost`

and

`127.0.0.1`

Try keeping both same.

Example:

Frontend:

`http://localhost:5173`

Backend:

`http://localhost:8000`

Then fetch:

`</> TypeScript`

Cleaner and avoids weird issues sometimes.



```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
Import "fastapi" could not be
resolvedPylancereportMissingImports
```

```
app = FastAPI()
```

```
origins = ["http://localhost:5173"]
```

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methos=["*"],
```

```

        allow_headers=["*"],
    )

```

```

todos = []

```

```

# @app.get("/")
# def home():
#     return {"message": "Backend working!"}

```

```

@app.get("/todos")
def get_todos():
    print("todos:", todos)
    return todos

```

```

@app.post("/todos")
def add_todo(todo: dict):
    todos.append(todo)
    print("todos:", todos)
    return {"message": "Todo added"}

```

```

File "D:\Project\2026\python\todo-react-
python\backend\venv\Lib\site-
packages\uvicorn\middleware\proxy_headers.py", line 56,
in __call__
    return await self.app(scope, receive, send)

```

```

^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "D:\Project\2026\python\todo-react-
python\backend\venv\Lib\site-
packages\fastapi\applications.py", line 1159, in __call__
    await super().__call__(scope, receive, send)

```

```

File "D:\Project\2026\python\todo-react-
python\backend\venv\Lib\site-
packages\starlette\applications.py", line 89, in __call__
    self.middleware_stack = self.build_middleware_stack()

```

```

^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "D:\Project\2026\python\todo-react-
python\backend\venv\Lib\site-
packages\fastapi\applications.py", line 1065, in
build_middleware_stack

```

```
app = cls(app, *args, **kwargs)
```

```
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

TypeError: CORSMiddleware.__init__() got an unexpected keyword argument 'allow_methos'

INFO: 127.0.0.1:56589 - "OPTIONS /todos HTTP/1.1" 500 Internal Server Error

You only have a TYPO error.

You wrote:

</> Python



▶ Run

Correct spelling:

</> Python



▶ Run

Missing the d .

Correct Version

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

origins = ["http://localhost:5173"]

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

todos = []
```



```
@app.get("/todos")
def get_todos():
    print("todos:", todos)
    return todos

@app.post("/todos")
def add_todo(todo: dict):
    todos.append(todo)
    print("todos:", todos)
    return {"message": "Todo added"}
```

About This Error

TypeError: CORSMiddleware.__init__() got an unexpected keyword argument 'allow_methos' 

Python is saying:

"I don't know what allow_methos means."

Because:

- allow_methos 
- allow_methods 

About This Warning

Import "fastapi" could not be resolved 

This usually means:

VS Code is NOT using your virtual environment.

Fix VS Code Python Interpreter

Press:

Ctrl + Shift + P 

Then:

Python: Select Interpreter



Choose:

backend/venv/Scripts/python.exe



or your venv interpreter.

If FastAPI Not Installed

Activate venv first:

Windows PowerShell

</> Bash



Then install:

</> Bash



Restart Server

After fixing:

</> Bash



Then:

</> Bash



Your Flow After Fix

Frontend React:

- sends request

Backend FastAPI:

- accepts request
- CORS allows frontend
- returns data

Everything should work now.



↳ backend/venv/Scripts/python.exe

still not showing?

That means VS Code is not detecting your virtual environment automatically.

Do this manually.

Step 1 — Check venv Exists

Inside your backend folder you should have:

```
backend/  
├─ venv/  
└─ main.py
```



If `venv` folder is missing, create it.

Step 2 — Create Virtual Environment

Open terminal inside backend folder:

```
</> Bash
```



This creates:

```
venv/
```



Step 3 — Activate venv

PowerShell

```
</> Bash
```



```
.\venv\Scripts\Activate
```

If successful:

```
(venv) PS D:\Project...
```



appears in terminal.

Step 4 — Install Packages

```
</> Bash
```



```
pip install fastapi uvicorn
```

Step 5 — Select Interpreter in VS Code

Press:

```
Ctrl + Shift + P
```



Search:

```
Python: Select Interpreter
```



If not showing:

- click:

```
Enter interpreter path
```



Then:

Find...



Now manually choose:

backend/venv/Scripts/python.exe



IMPORTANT

You must select:

python.exe



NOT:

- activate
- activate.bat
- pip.exe

After Selecting

Bottom-right VS Code should show something like:

Python 3.x.x (venv)



Then this warning disappears:

Import "fastapi" could not be resolved



Quick Check

Run:

`</> Bash`



`pip list`

If you see:

fastapi



```
uvicorn
```

then everything is installed correctly.

